Proyecto núcleo 2

Karol Cardenas – Camilo Méndez

1. Actores del sistema (Stakeholders)

Conforme a Hu (2023), el levantamiento de requerimientos parte de identificar los actores que interactúan con el sistema, ya que sus necesidades son la fuente primaria de los requerimientos funcionales.

Actor	Descripción
Ciudadano	Usuario final de la plataforma. Realiza trámites, pagos en línea, participa en foros y comunidades de la Ciudad Digital.
Funcionario municipal	Empleado del municipio. Administra contenidos del portal, gestiona trámites internos, publica información oficial.
Administrador del sistema	Responsable técnico. Gestiona roles, permisos, configuración global y monitoreo de la plataforma.
Sistema externo gubernamental	Entidades de gobierno externas que se integran con la plataforma mediante mecanismos de interoperabilidad.

levantamiento de requerimientos:

El levantamiento se elaboró con base en la metodología descrita por Hu (2023) en el libro guía de la asignatura, en este libro se establece que los requerimientos deben cumplir con lo siguiente:

1. Identificarse a partir de los actores del sistema
2. Separarse claramente en funcionales y no funcionales
3. Priorizarse según si criticidad para el funcionamiento del sistema

ID	Descripción	Actor principal	Módulo	Prioridad
RF-01	El sistema debe permitir el registro de usuarios ciudadanos con datos básicos de identificación.	Ciudadano	Gestión de usuarios	**Alta**

ID	Descripción	Actor principal	Módulo	Prioridad
RF-02	El sistema debe permitir el inicio de sesión mediante autenticación segura (usuario/contraseña).	Ciudadano / Funcionario	Gestión de usuarios	**Alta**
RF-03	El sistema debe permitir la gestión del perfil de usuario: datos personales, foto e información de contacto.	Ciudadano	Gestión de usuarios	**Media**
RF-04	El sistema debe permitir a los administradores gestionar roles y permisos diferenciados (ciudadano / funcionario / administrador).	Administrador	Gestión de usuarios	**Alta**
RF-05	El sistema debe permitir la publicación de noticias, eventos y comunicados oficiales del municipio.	Funcionario	Portal municipal	**Alta**
RF-06	El sistema debe permitir la administración de contenidos del portal por parte de funcionarios autorizados (back-office).	Funcionario / Admin	Portal municipal	**Alta**
RF-07	El sistema debe permitir la publicación de información de transparencia, rendición de cuentas e indicadores de gestión.	Funcionario	Portal municipal	**Alta**
RF-08	El sistema debe permitir que los ciudadanos realicen trámites en línea: solicitud, envío de documentos y confirmación.	Ciudadano	Ventanilla única	**Alta**
RF-09	El sistema debe permitir a los ciudadanos consultar el catálogo de	Ciudadano	Ventanilla única	**Media**

ID	Descripción	Actor principal	Módulo	Prioridad
	trámites disponibles en su municipio.			
RF-10	El sistema debe permitir a los ciudadanos consultar el estado de sus trámites en tiempo real.	Ciudadano	Ventanilla única	**Alta**
RF-11	El sistema debe enviar notificaciones automáticas al ciudadano sobre cambios en el estado del trámite.	Ciudadano	Ventanilla única	**Alta**
RF-12	El sistema debe permitir realizar pagos en línea de trámites municipales mediante pasarela de pagos integrada.	Ciudadano	Pagos en línea	**Alta**
RF-13	El sistema debe generar y permitir la descarga de comprobantes de pago en formato digital.	Ciudadano	Pagos en línea	**Alta**
RF-14	El sistema debe permitir a los ciudadanos participar en foros, comunidades virtuales y wikis.	Ciudadano	Ciudad digital	**Alta**
RF-15	El sistema debe permitir la publicación y gestión de blogs y contenidos multimedia de interés ciudadano.	Ciudadano	Ciudad digital	**Media**
RF-16	El sistema debe permitir la creación, publicación y gestión de eventos ciudadanos y gubernamentales.	Ciudadano / Funcionario	Ciudad digital	**Media**
RF-16b	El sistema debe incluir un directorio de negocios locales y vitrinas	Ciudadano / Empresas	Ciudad digital	**Media**

ID	Descripción	Actor principal	Módulo	Prioridad
	comerciales para PYMES del municipio.			
RF-17	El sistema debe permitir la integración con sistemas de otras entidades gubernamentales cuando se requiera.	Sistema externo	Interoperabilidad	**Media**
RF-18	El sistema debe permitir la generación de reportes administrativos e indicadores KPI para la gestión municipal.	Funcionario / Admin	Interoperabilidad	**Media**

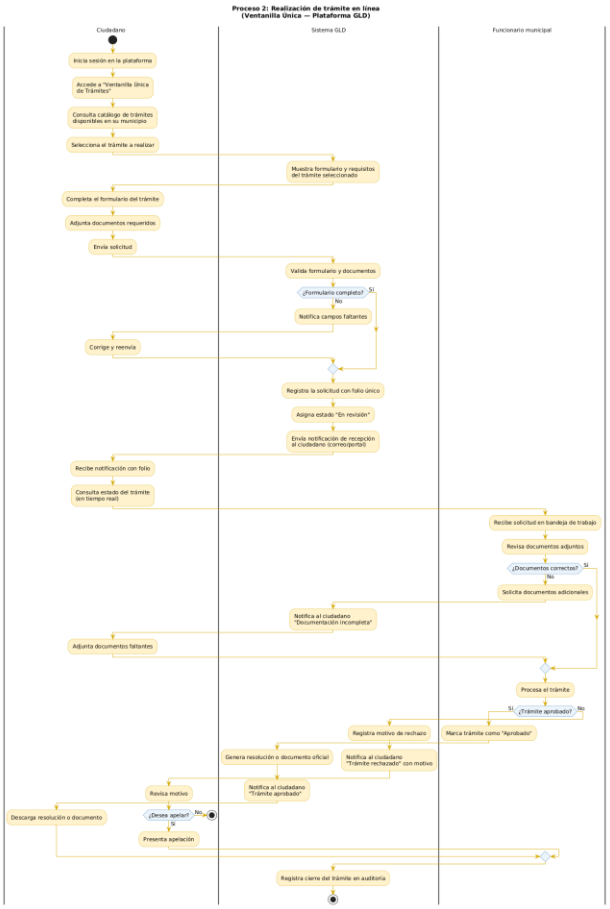
BPMN

Los procesos deben modelarse por unidades de negocio significativo, no todo el sistema debe procesarse, Un solo BPMN general para toda la plataforma del GLD tendría demasiadas actividades, por lo que sería completamente ilegible.

La práctica correcta es modelar un solo proceso por diagrama que responda una pregunta concreta

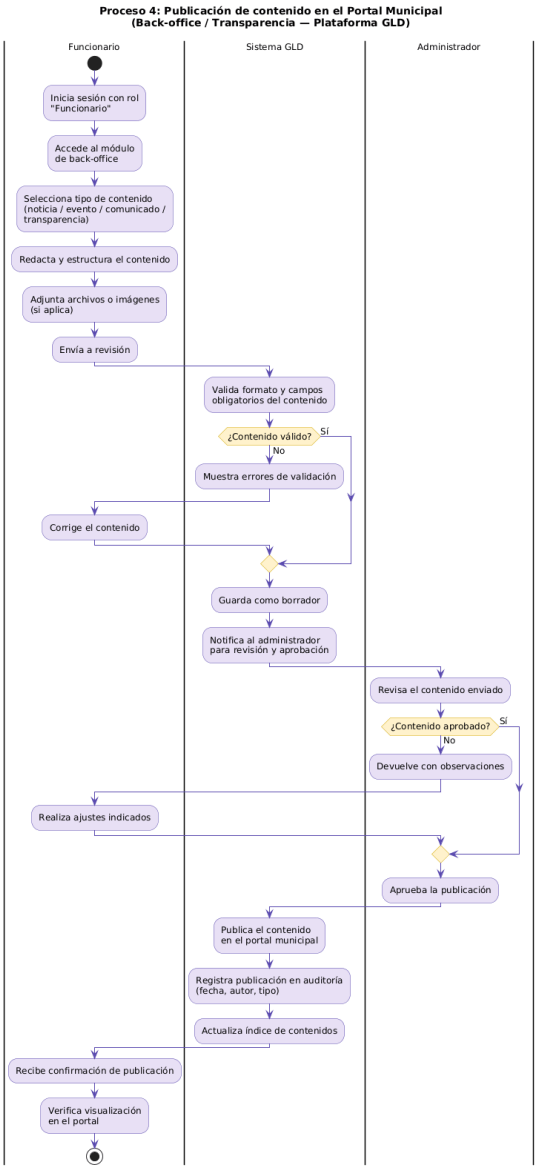
BPMN del proceso de función de tramite

Este es el proceso más crítico del sistema porque es la razón principal de la existencia del mismo, en este paso eliminamos los tramites presenciales, por ello el diagrama tiene tres carriles, ciudadano, sistema GLD y el funcionario, el flujo inicia cuando el ciudadano busca un trámite en el catálogo, completa y envía, el sistema valida y le asigna un folio único, el cual es el mecanismo de trazabilidad. Luego el funcionario toma el control y decide si se aprueba o no (para ello puede volver a pedirle información al usuario) en caso de rechazarse el ciudadano puede apelar y cada cambio de estado envía una notificación cumpliendo con el RF-11



2. **proceso de publicación de contenido**

Acá modelamos el back-office del portal comandando con el funcionario que crea y publica las noticias, eventos, comunicados o información de transparencia del estado, cuanta con tres carriles: funcionario, sistema y admin. Podemos resaltar de este flujo el paso de aprobación, el funcionario redacta, pero para evitar cualquier inconveniente debemos de tener supervisión que aceptara o rechazara la publicación conforme se vea pertinente, proceso fundamental para el RF-07 pues acá haremos todo para mantener transparencia



3. participación ciudadana

Este proceso es el módulo social de la plataforma, el ciudadano puede crear un nuevo tema en un foro o responder a uno existente, por eso, hay decisiones lógicas según el contenido

que tenga cada foro.

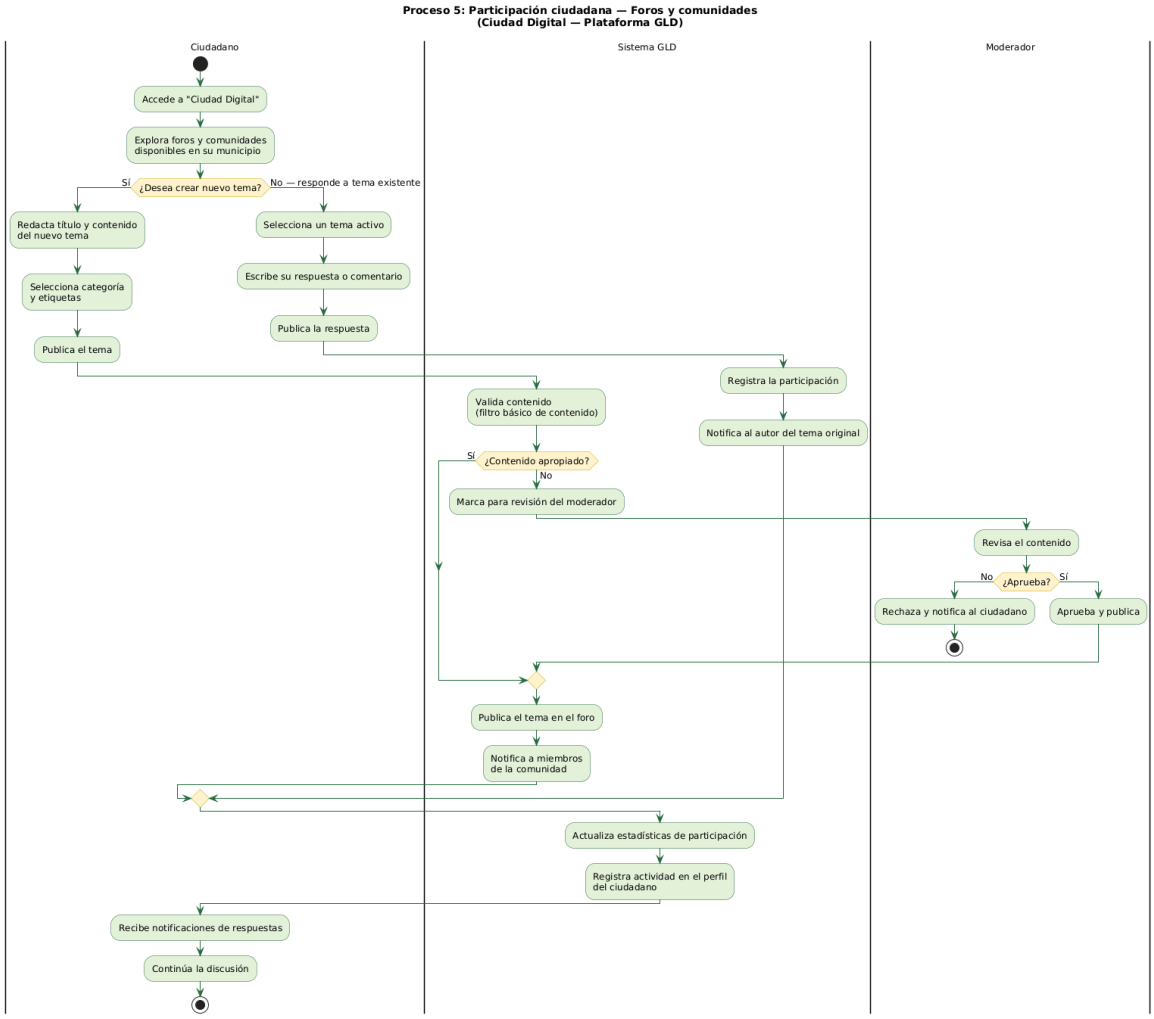


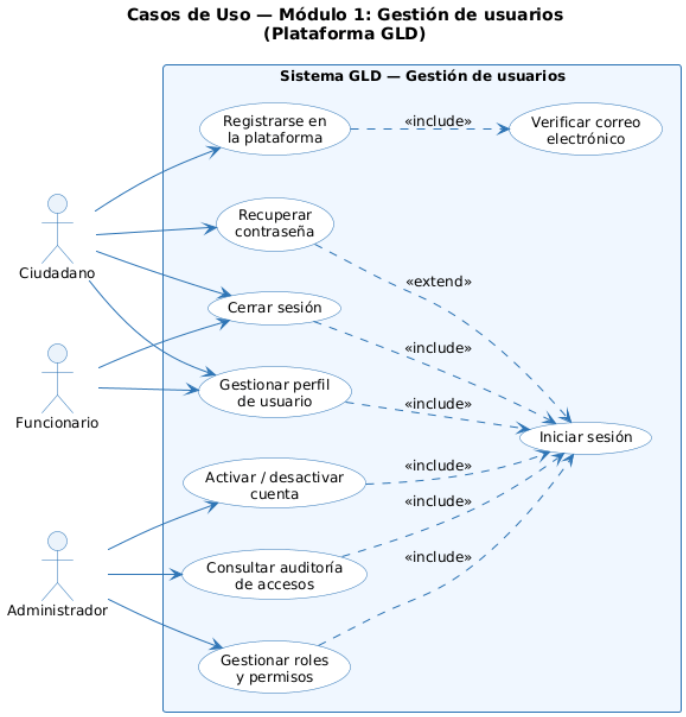
Diagrama de casos de uso

Según Hu estos diagramas debe tener tres elementos principales:

1. **Actores:** Quien usa el sistema
2. **Casos de uso:** Que hace cada actor
3. **Relaciones:** Como interactúan

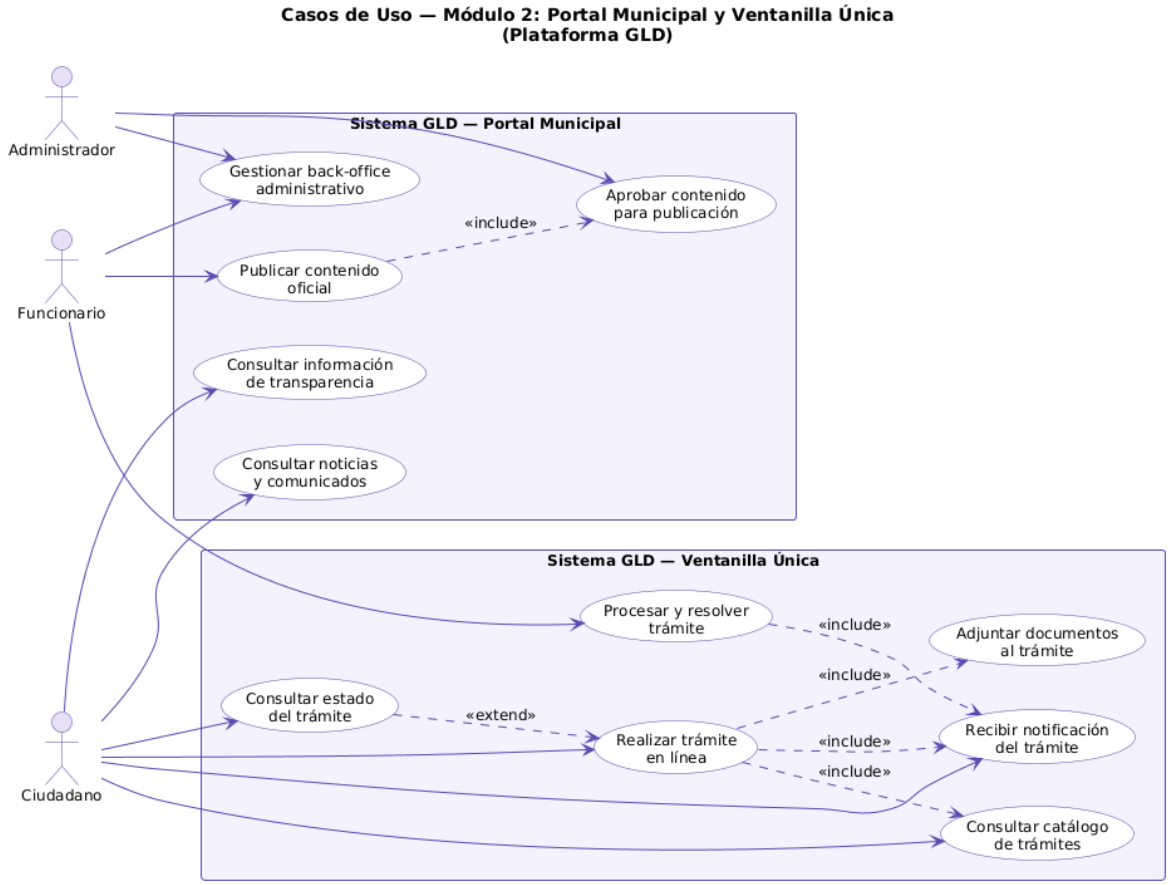
Las relaciones se basan en dos include (caso de uso que siempre llama a otro obligatoriamente) y exclude (Opcionalmente invocados), además de que los casos de uso deben modelarse desde la perspectiva del valor que aportan al actor, no desde la perspectiva del sistema

Diagrama 1



Cubrimos el ciclo de vida completo de una cuenta en la plataforma. Los tres actores tienen acceso según su función (primer paso de la triada AAA de seguridad) el ciudadano gestiona su propia cuenta, el funcionario tiene acceso operativo básico y el admin tiene control total de roles y permisos

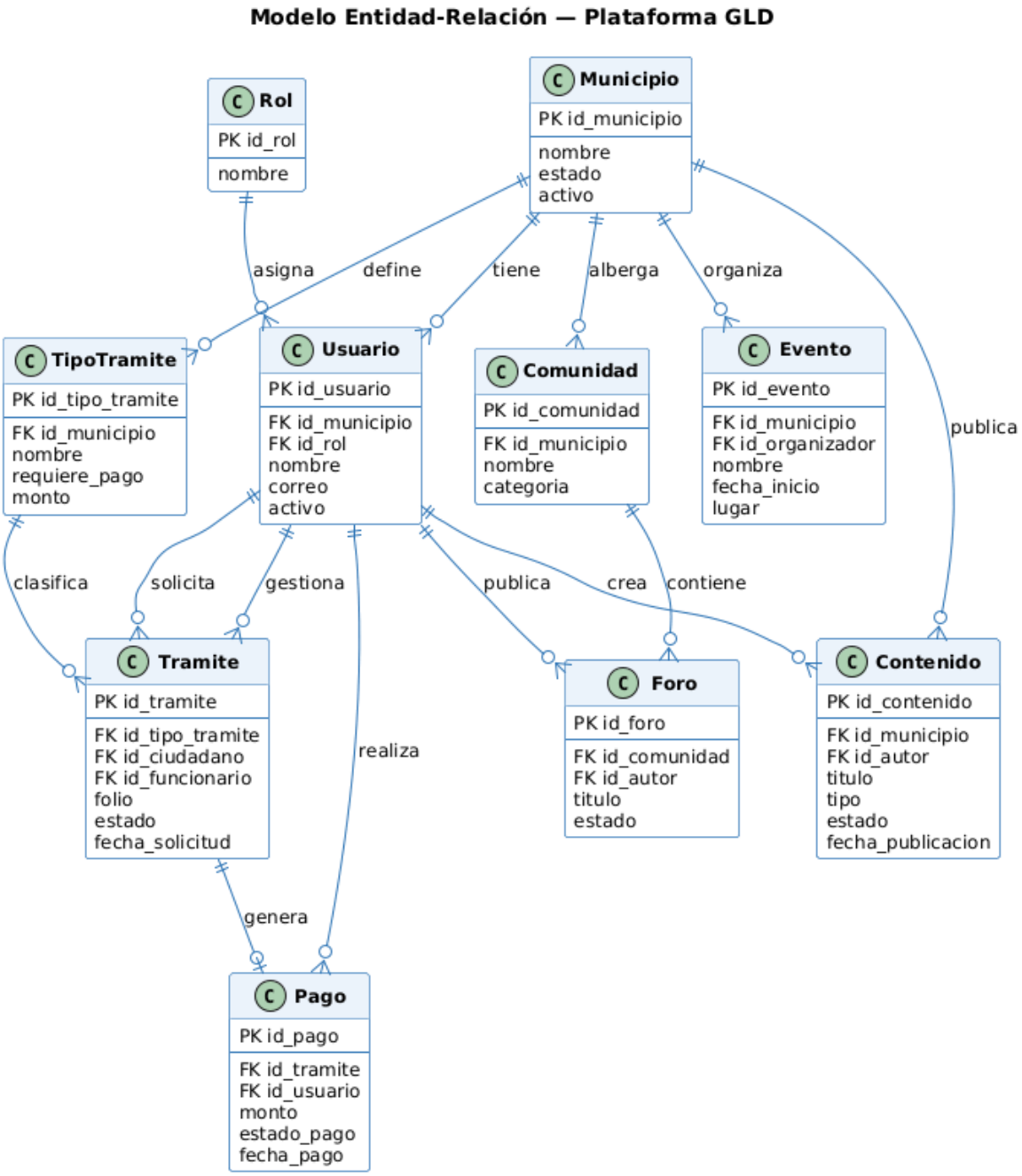
Diagrama 2



En este diagrama agrupamos dos sistemas que comparten actores, el ciudadano consume información del portal, pero a su vez es el actor principal de la ventanilla, para hacer una publicación obligatoriamente se tiene que aprobar por ello el <<include>>

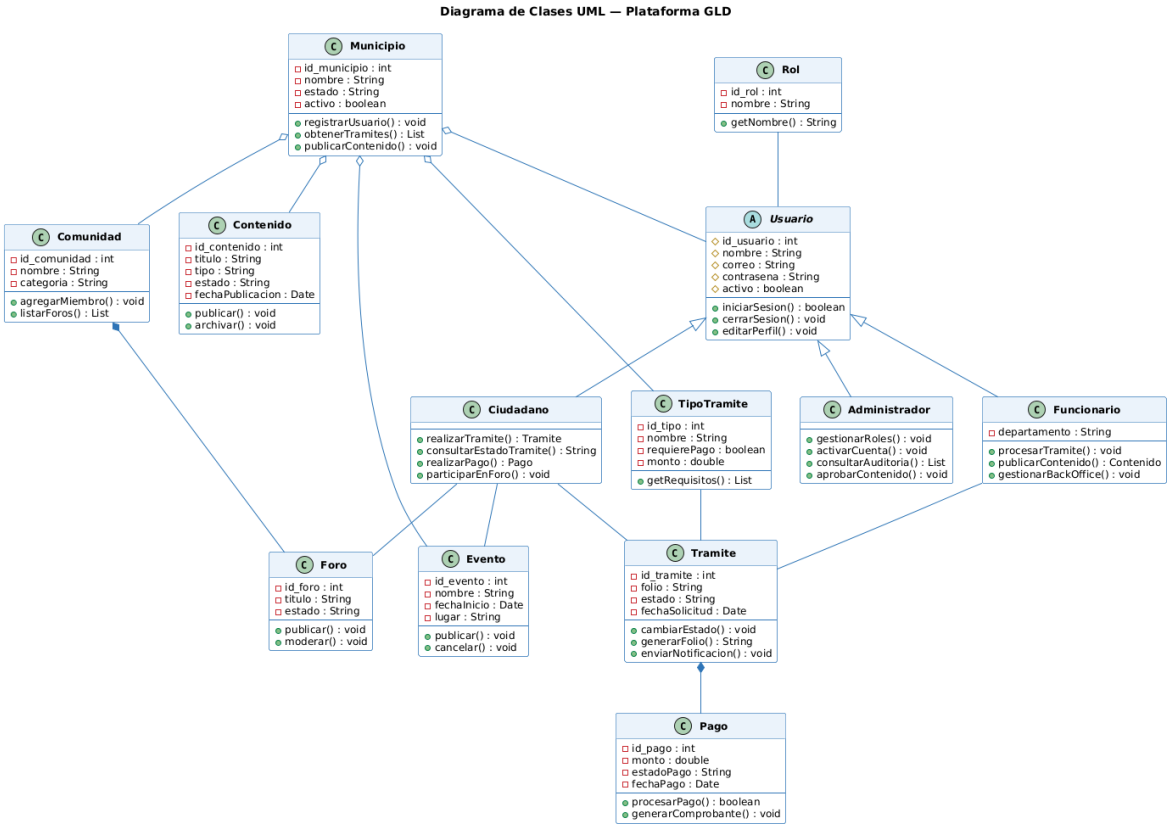
4. Modelo entidad relacion

En su libro Hu establece que el MER debe venir directamente de los requerimientos funcionales, donde cada entidad representa un concepto que el sistema debe persistir, estos conceptos interactúan entre sí por lo tanto llevan relaciones



Mirando desde arriba hacia abajo notamos que la parte izquierda y el centro del modelo corresponden al portal municipal y a la ventanilla única, la cual es la funcionalidad del gobierno digital, mientras que el lado derecho, corresponde a la ciudad digital, acá lo primordial es la funcionalidad de participación y comunicación ciudadana, el municipio y el usuario son el puente entre ambas áreas pues todo el sistema tiene al menos un municipio y un usuario

5. Diagrama de clases



En este diagrama respondemos la pregunta de que sabe y que puede hacer cada objeto, apareciendo la encapsulación de atributos, métodos y la herencia

Herencia:

Como estamos en POO debemos tener presente este tema, el usuario es una clase abstracta por lo tanto no la podemos instanciar directamente, solo existe como base, por lo tanto, las tres clases heredan de él, sus métodos y cada una agrega su propio comportamiento especifico, el ciudadano puede procesar tramites y publicar contenido, al administrador gestiona roles y aprueba las publicaciones.

Cada clase tiene métodos que reflejan directamente los casos de uso del sistema como "tramite.generarFolio() Rf-08", o "pago.generarComprobante() Rf-13".

6. Arquitectura del Sistema

Justificación de tecnologías

Se realiza la selección de tecnologías respondiendo a los requerimientos no funcionales definidos, en particular RNF-1 que corresponde a la disponibilidad 99%, RNF-9 hace referencia a la escalabilidad multi-municipio, RNF-10 corresponde a código abierto y RNF-3 indica un control de acceso basado en roles. Todas las herramientas que seleccionamos corresponden a una reducción de costos, ya que son de licencia libre, sin costo de

licenciamiento, y con amplia documentación y comunidad activa, lo que las hace viables para una implementación productiva.

Capa	Tecnología	Justificación
Frontend	React.js + Tailwind CSS	Es un framework SPA de código abierto (MIT). Tiene amplio soporte de componentes accesibles, rendimiento en cliente que contribuye a RNF-8 respuesta < 3 s. Permite interfaz responsive para RNF-11 móviles y tabletas.
Backend	Node.js + Express.js	Runtime JavaScript asíncrono, no bloqueante, es ideal para múltiples solicitudes concurrentes de ciudadanos. Licencia MIT. Un solo lenguaje que es JavaScript en frontend y backend reduce la curva de aprendizaje para el equipo.
Base de datos	PostgreSQL	Motor relacional de código abierto que soporta directamente el MER definido. Permite multi-tenancy mediante esquemas separados por municipio (RNF-9). Licencia PostgreSQL libre.
Caché	Redis	Almacenamiento en memoria para sesiones de usuario y respuestas frecuentes como el catálogo de trámites, noticias, contribuyendo concretamente al RNF-8.
Autenticación	JWT + bcrypt	Tokens sin estado que escalan horizontalmente sin sesiones en servidor. bcrypt se va a utilizar para hash seguro de contraseñas. Cumple RNF-3 y RNF-4.
Notificaciones	Nodemailer + Socket.io	Nodemailer para correos transaccionales de estado de trámite RF-11. Socket.io para actualizaciones en tiempo real del estado RF-10. Son open source.
Pasarela de pagos	Wompi (sandbox)	Wompi ofrece un entorno sandbox gratuito para desarrollo y pruebas. API REST simple de integrar.
Hosting / Despliegue	Railway (backend + BD) + Vercel (frontend)	Permiten despliegue con un solo repositorio Git.
Control de versiones	GitHub	Es para repositorios públicos y privados. Integración directa con Railway y Vercel para CI/CD automático.

Justificación de patrones de diseño y arquitectónicos

Patrón MVC: Aplicado en el backend con Express.js. Separa la lógica de negocio que vienen siendo los controllers, el acceso a datos que son los models y la respuesta al cliente en formato JSON consumido por React, lo que facilita el mantenimiento, las pruebas unitarias y la incorporación de nuevos integrantes al equipo de desarrollo, dando cumplimiento directo a RNF-10.

Patrón Repository: Abstrae las consultas a PostgreSQL de la lógica de negocio. Cada entidad principal del MER como lo son Tramite, Usuario, Contenido, Pago, tiene su propio repositorio. Permitiendo modificar o reemplazar la capa de persistencia sin impactar los servicios de negocio, protegiendo la mantenibilidad del sistema a largo plazo.

Patrón Observer: Implementado para el módulo de notificaciones RF-11. Cuando el estado de un trámite cambia, el servicio de notificaciones se dispara automáticamente como suscriptor del evento, sin que el módulo de trámites tenga conocimiento directo del módulo de notificaciones. Esto reduce el acoplamiento entre módulos y facilita agregar nuevos tipos de notificación en el futuro.

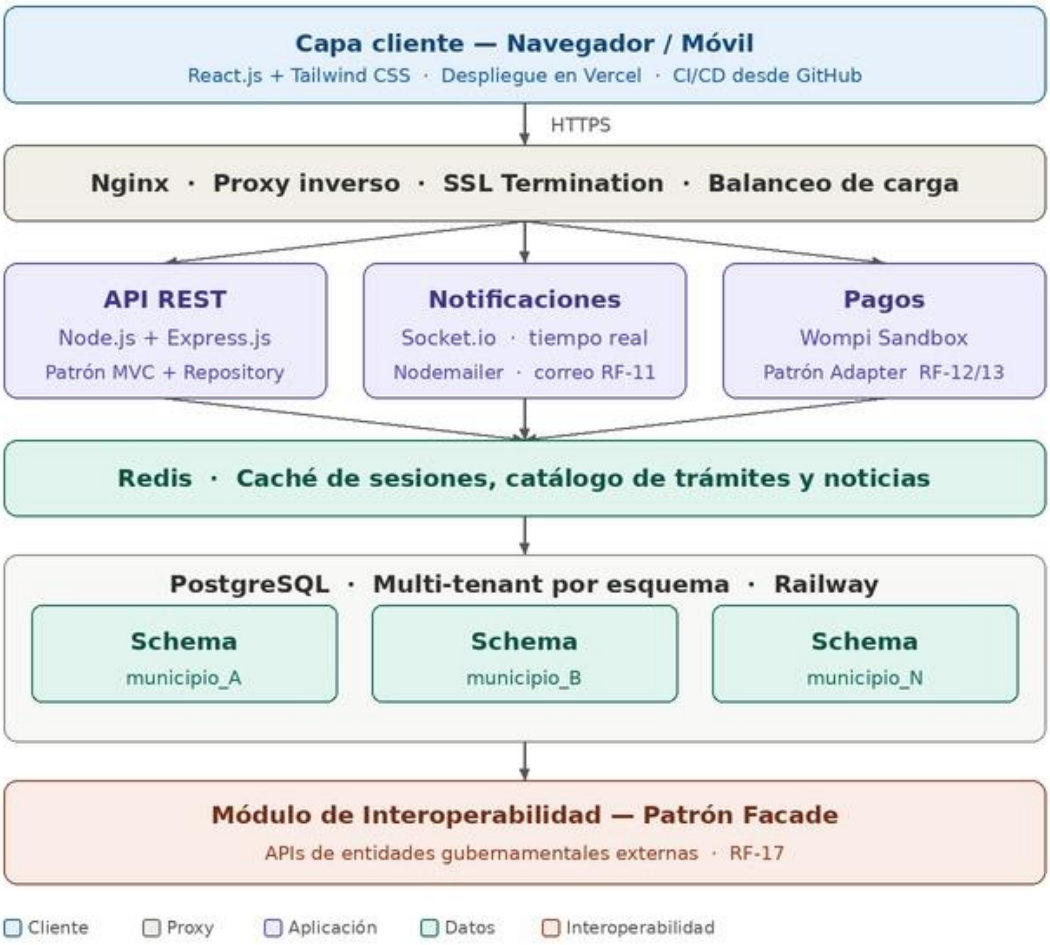
Patrón Adapter: Aplicado en el módulo de pagos RF-12, RF-13. El adaptador de Wompi traduce la interfaz del proveedor externo a la interfaz interna de la plataforma. Si en el futuro se cambia de pasarela de pagos, únicamente se reemplaza el adaptador sin modificar la lógica de negocio del sistema.

Patrón Facade: El módulo de interoperabilidad RF-17 muestra una fachada unificada hacia los sistemas gubernamentales externos, ocultando la complejidad y heterogeneidad de cada integración particular. El resto del sistema interactúa con una sola interfaz simple sin necesidad de conocer los detalles de cada entidad externa.

Arquitectura multi-tenant por esquema: Cada municipio suscrito a la plataforma SaaS opera en un esquema PostgreSQL separado. Garantizando el aislamiento completo de datos entre municipios RNF-9, RNF-4 y permite incorporar nuevos municipios sin afectar el funcionamiento de los que ya existen, permitiendo la escalabilidad del modelo de negocio propuesto.

Arquitectura Cliente-Servidor desacoplada (REST API): El frontend en React consume exclusivamente una API REST construida sobre Express. Este desacoplamiento permite que en el futuro se desarrolle una aplicación móvil nativa consumiendo la misma API sin modificar el backend, cumpliendo con RNF-11 y protegiendo la inversión del desarrollo realizado.

Diagrama de Arquitectura General


Capa cliente — Navegador / Móvil
React.js + Tailwind CSS · Despliegue en Vercel · CI/CD desde GitHub
HTTPS
Nginx · Proxy inverso · SSL Termination · Balanceo de carga
API REST
Node.js + Express.js
Patrón MVC + Repository
Notificaciones
Socket.io · tiempo real
Nodemailer · correo RF-11
Pagos
Wompi Sandbox
Patrón Adapter RF-12/13
Redis · Caché de sesiones, catálogo de trámites y noticias
PostgreSQL · Multi-tenant por esquema · Railway
Schema
municipio_A
Schema
municipio_B
Schema
municipio_N
Módulo de Interoperabilidad — Patrón Facade
APIs de entidades gubernamentales externas · RF-17
Cliente
Proxy
Aplicación
Datos
Interoperabilidad